UNIT–VI: I/O Systems

- **I/O Hardware**
- **Protection**
  - **Access Control List (ACL)**
  - **Capabilities**
- **Program Threats**

Overview

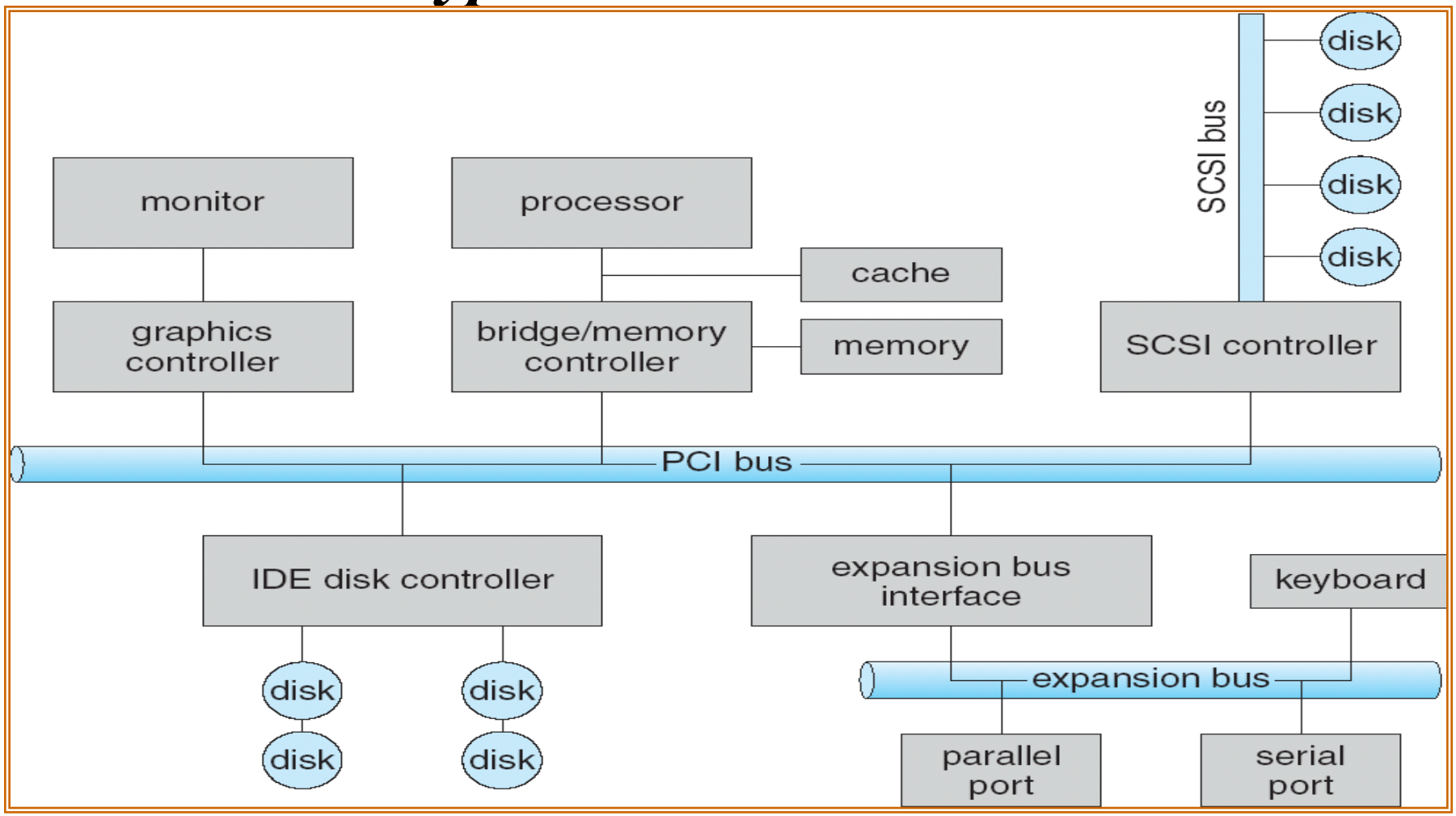
- The control of devices connected to the computer is a major concern of operating system designers.
- Various methods are needed to control them. These methods form the I/O subsystem of the kernel, which separates the rest of the kernel from the complexities of managing I/O devices.
- The device drivers present a uniform device-access interface to the I/O subsystem, much as system calls provide an interface between applications and the operating system.

I/O Hardware

- **Incredible variety of I/O Devices**
  - **Storage Devices – Disks, Tapes, …**
  - **Transmission Devices – Network cards, Modems**
  - **Human Interface Devices – Screen, Keyboard, Mouse**
  - **Other Specialised Devices – Steering of a Space shuttle**
- **Common Concepts**
  - **Port (Connection point)**
  - **Bus (Connection of Common Set of wires)**
  - **Controller (Host Adapter)**

    Operates a Port, Bus, or a Device.
- **I/O instructions Control Devices**
- **Devices have addresses, used by**
  - **Direct I/O instructions**
  - **Memory–Mapped I/O**

A Typical PC Bus Structure



- **PCI** **Peripheral Component Interconnect**
- **SCSI** **Small Computer System Interface**
- **IDE** **Integrated Development Environment**

Device I/O Port Locations on PCs (Partial)

I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)

I/O Port Registers

|<<

- ***Data–in Registers***
  Read by the host to get input

- ***Data–out Registers***
  Written by the host to send output

- ***Status Registers***
  Contain bits that can be read by the host

- ***Control Registers***
  Written by the host to Start a Command
  or to Change the Mode of a Device

I/O Hardware

- A device communicates with computer by sending signals over a cable or even through air. The device communicates with machine via a connection point(or port).
- **An I/O port and its registers**
- **A Controller**
- **A Bus(is a set of wires and rigidly defined protocol that specifies a set of messages that can be sent on wires).**
- **Daisy chain**
- **The PCI bus connects the processor-memory subsystem to the fast devices and an expansion bus that connects slow devices like keyboard and serial and parallel ports.**

Polling

|<<

- **Uses Handshake protocol**

  Eg. We assume that two bits are used to coordinate between controller and the host.

  The controller indicates its state through the *busy* bit in the status register. The controller sets the *busy* bit when it is busy working and clears the *busy* bit when it is ready to accept next command.

- **Determines State of Device**
  - **Command–Ready** (hosts signals its wish with this bit set in *command* register and when the command is available for the controller to execute).

  For this example, the host writes output through a port, coordinating with the controller by handshaking as follows.

  1. The host repeatedly reads the *busy* bit until that bit becomes clear.
  2. The host sets the *write* bit in the *command* register and writes a byte into *data-out* register.
  3. The host sets the *command-ready* bit.
  4. When the controller notices that the *command-ready* bit set, it sets the *busy* bit.
  5. The controller reads the command register and sees the *write* command. It reads the *data-out* register to get the byte and does I/O to the device

Contd..

6. The controller clears the *command-ready* bit, clears the *error* bit in the status register to indicate that the device I/O succeeded, and clears the *busy* bit to indicate that it is finished.

This loop is repeated for each byte.

- In step 1, the host is **Busy–waiting or polling:** It is in a loop, reading the *status* register over and over until the *busy* bit becomes clear.
  - **To wait for I/O from Device**
- Basic operation of polling is efficient. But it becomes inefficient when it attempted repeatedly yet rarely finds a device to be ready for the service.
- So, its better to arrange a hardware controller to notify the CPU when the device becomes ready for service- called an interrupt.

Interrupts

- **When interrupt I/O is used, the CPU is free to ignore the I/O module until the interrupt signal is received.**

- **CPU Interrupt–request line**
  - Triggered by I/O Device

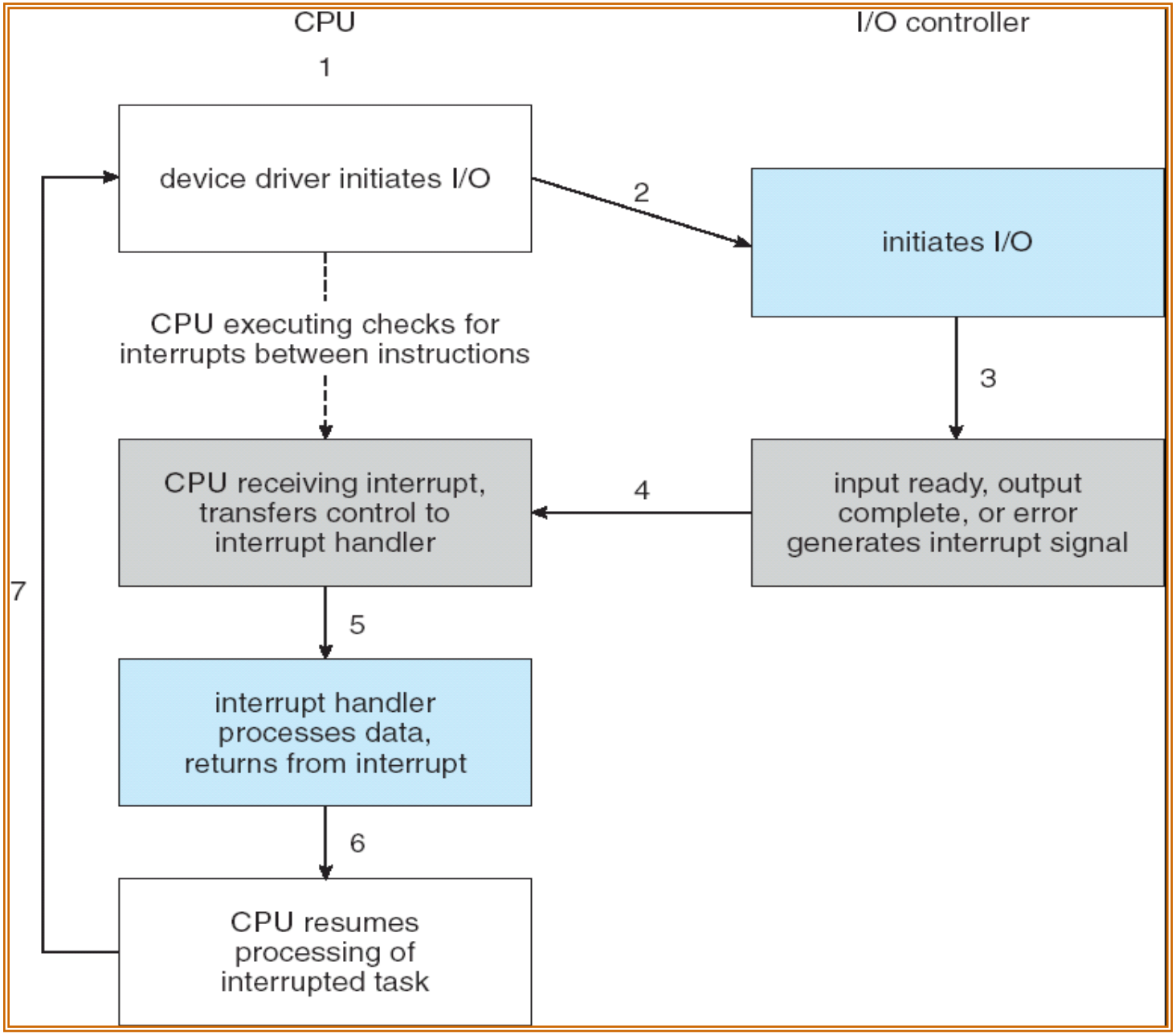
- **Interrupt Handler**
  - Receives Interrupts

- **Most CPUs have two interrupt request lines.**
  - Maskable and Nonmaskable
- Maskable to ignore or delay some interrupts
- Non maskable, reserved for events such as unrecoverable memory errors.

- Interrupt vector to dispatch interrupt to correct handler
  - Based on priority
  - Some unmaskable

  Interrupt mechanism also used for exceptions

Interrupt–Driven I/O Cycle



Polling vs. Interrupt-Driven I/O

- Polling requires code to loop until device is ready
  - Consumes *lots* of CPU cycles
  - Can provide quick response (guaranteed delay)
- Interrupts don't require code to loop until the device is ready
  - Device interrupts processor when it needs attention
  - Code can go off and do other things
  - Interrupts can happen at any time
    - Requires careful coding to make sure other programs (or your own) don't get messed up

Direct Memory Access

- For a device that does large transfers, such as a disk drive, it seems wasteful to use an expensive general-purpose processor to watch status bits and to feed data into controller register one byte at a time-a process termed Programmed I/O(PIO).
- A special control unit may be provided to allow transfer of a block of data directly between an external device and the main memory, without continuous intervention by the processor. This is called Direct Memory Access(DMA).

- Offloading of the work to a DMA Controller for large transfers
- Used to avoid *Programmed I/O*
  - for large Data Movement
- Requires *DMA Controller*
- Bypasses *CPU*
  - to transfer Data directly between I/O Device and Memory

Contd..

- To initiate a DMA transfer, the host writes a DMA command block into memory. This block contains a pointer to the source of a transfer, a pointer to the destination of the transfer, and a count of the number of bytes to be transferred.
- The CPU writes the address of this command block to the DMA controller, then goes on with other work.

Contd..

- The DMA controller proceeds to operate the memory bus directly, placing the addresses on the bus to perform transfers without the help of the main CPU.
- A simple DMA controller is a standard component in PCs, and bus-mastering I/O boards for the PC usually contain their own high-speed DMA hardware.

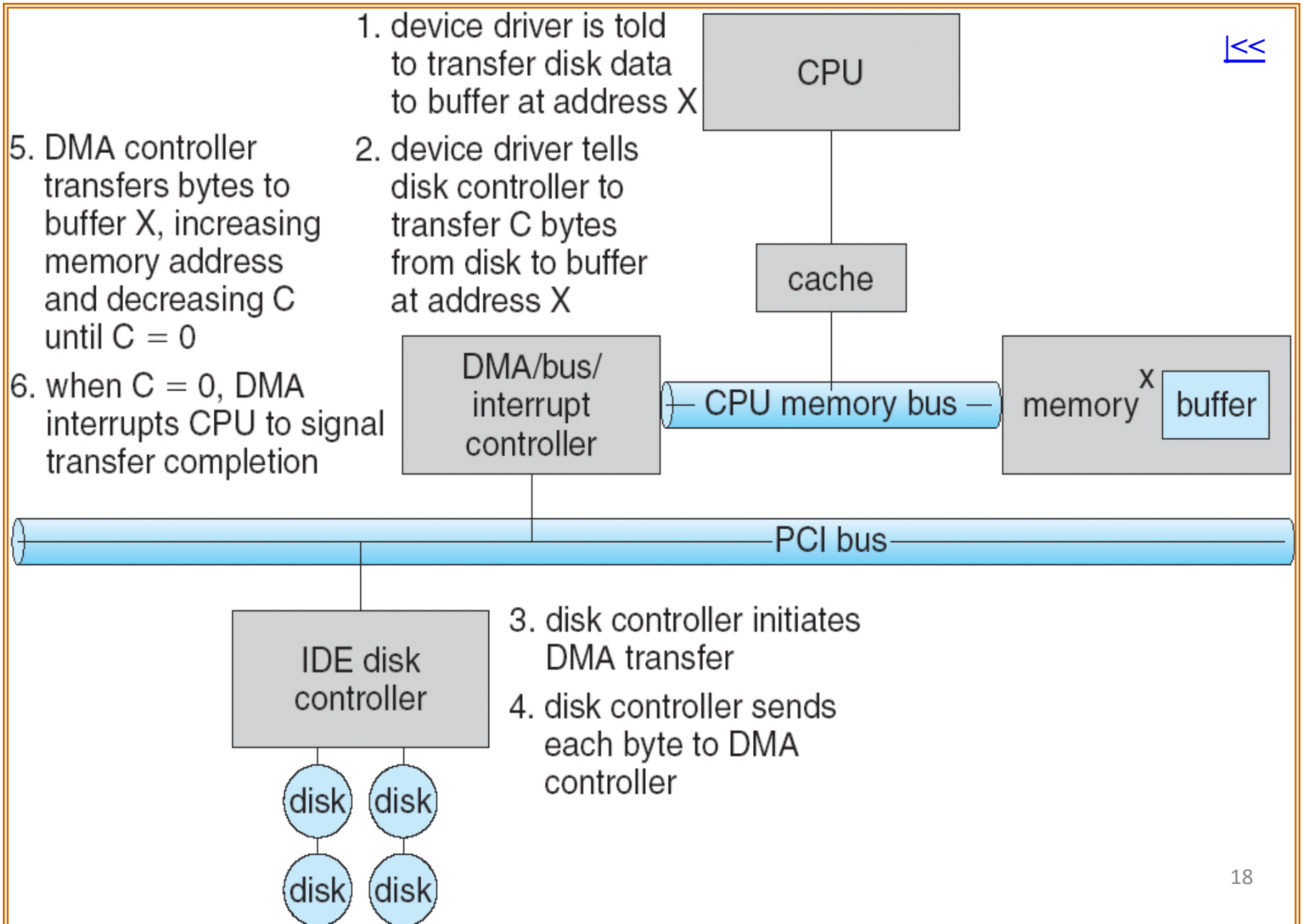
Contd..

- Handshaking between the DMA controller and the device controller is performed via a pair of wires called DMA-request and DMA-acknowledge.
- The device controller places a signal on the DMA-request wire when a word of data is available for the transfer.
- This signal causes the DMA controller to seize the memory bus, to place the desired address on the memory-address wires, and to place a signal on DMA-acknowledge wire.

Contd..

- When the device controller receives the DMA-acknowledge signal, it transfers the word of data to memory and removes the DMA-request signal.
- When the entire transfer is finished, the DMA controller interrupts the CPU.

Six Step Process to Perform DMA Transfer



Protection

- Goals of Protection
- Principles of Protection
- Access Matrix
- Implementation of Access Matrix
- Access Control
- Revocation of Access Rights
- Capability-Based Systems

Objectives

- Discuss the goals and principles of protection in a modern computer system
- Explain how protection domains combined with an access matrix are used to specify the resources a process may access
- Examine capability and language-based protection systems

Goals of Protection

- Operating system consists of a collection of objects, hardware or software
- Each object has a unique name and can be accessed through a well-defined set of operations.
- Protection problem - ensure that each object is accessed correctly and only by those processes that are allowed to do so.

Principles of Protection

- Guiding principle – principle of least privilege
  - Programs, users and systems should be given just enough privileges to perform their tasks

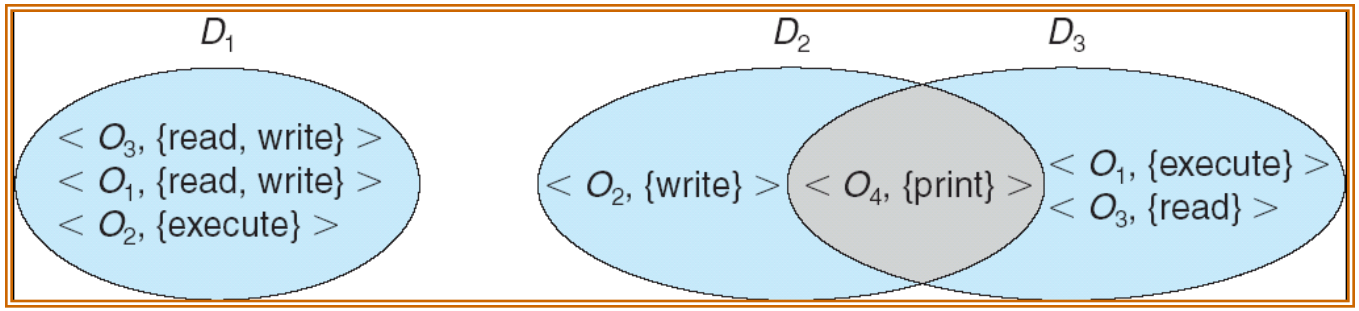
➢ Mechanism for Controlling the Access of Programs, Processes, or users to the resources defined by a Computer System.

➢ Must Provide a means for Specifying the Controls to be imposed, together with a means of enforcement.

Domain Structure

- Access-right = <*object-name*, *rights-set*> where *rights-set* is a subset of all valid operations that can be performed on the object.

- Domain = set of access-rights



Access Matrix

- View protection as a matrix (*access matrix*)

- Rows represent domains
- Columns represent objects

- *Access(i, j)* is the set of operations that a process executing in $Domain_i$ can invoke on $Object_j$

Access Matrix

object / domain	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Use of Access Matrix

- If a process in Domain D_i tries to do "op" on object O_j, then "op" must be in the access matrix.

- Can be expanded to dynamic protection.
  - Operations to add, delete access rights.
  - Special access rights:
    - *owner of* O_i
    - *copy op from* O_i *to* O_j
    - *control –* D_i *can modify* D_j *access rights*
    - *transfer – switch from domain* D_i *to* D_j

Use of Access Matrix (Cont.)

- Access matrix design separates mechanism from policy.
  - Mechanism
    - Operating system provides access-matrix + rules.
    - It ensures that the matrix is only manipulated by authorized agents and that rules are strictly enforced.
  - Policy
    - User dictates policy.
    - Who can access what object and in what mode.

Implementation of Access Matrix

- Each column = Access-control list for one object
  Defines who can perform what operation.
  Object i (O_i) Access Control List

  Domain 1 = Read, Write
  Domain 2 = Read
  Domain 3 = Read

  ⋮

- Each Row = Capability List (like a key)
  Fore each domain, what operations allowed on what objects.

  Object 1 – Read
  Object 4 – Read, Write, Execute
  Object 5 – Read, Write, Delete, Copy

Access Matrix of Figure A With Domains as Objects

domain \ object	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

Figure B

Access Matrix with *Copy* Rights

object / domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

object / domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

(b)

Access Matrix With *Owner* Rights

object / domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		

(a)

object / domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write

(b)

Modified access matrix

object / domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch control
D_3		read	execute					
D_4	write		write		switch			

Access Control List (ACL)

|<<

- **Associated with Each File and Directory.**
- **Specifies User names and the types of Access allowed for Each User.**
- **When User requests access to a particular file, the Operating System checks the access list associated with that file.**
- **If that user is listed for the requested access, the access is allowed.**
- **Otherwise, a Protection Violation occurs, and the user job is denied access to the file.**
- **Main problem is Length.**

Capabilities

|<<

- **All access to objects is done thru Capabilities,**
  and Capabilities provide the *only* means of accessing Objects.
- **In such a System, every Program holds a Set of Capabilities.**

  If Program A holds a Capability to talk to Program B,
  then the two Programs can grant Capabilities to each other.
- **In most Capability Systems,**
  a Program can hold an infinite no. of Capabilities.
- **A better Design allows each Program to hold**
  a fixed no. of Capabilities,
  and provides a means for Storing additional Capabilities
  if they are needed.
- **The only way to obtain Capabilities**
  is to have them granted as a result of Some Communication.

Implementation of Access Matrix

- Generally, a sparse matrix
- Option 1 – Global table
  - Store ordered triples < *domain, object, rights-set* > in table
  - A requested operation M on object O_j within domain D_i -> search table for < D_i, O_j, R_k >
    - with $M \in R_k$
  - But table could be large -> won't fit in main memory
  - Difficult to group objects (consider an object that all domains can read)

- Option 2 – Access lists for objects
  - Each column implemented as an access list for one object
  - Resulting per-object list consists of ordered pairs < *domain, rights-set* > defining all domains with non-empty set of access rights for the object
  - Easily extended to contain default set -> If $M \in$ default set, also allow access

- Each column = Access-control list for one object
  Defines who can perform what operation

  Domain 1 = Read, Write
  Domain 2 = Read
  Domain 3 = Read

- Each Row = Capability List (like a key)
  For each domain, what operations allowed on what objects

  Object F1 – Read
  Object F4 – Read, Write, Execute
  Object F5 – Read, Write, Delete, Copy

Implementation of Access Matrix (Cont.)

- Option 3 – Capability list for domains
  - Instead of object-based, list is domain based
  - **Capability list** for domain is list of objects together with operations allows on them
  - Object represented by its name or address, called a **capability**
  - Execute operation M on object O_j, process requests operation and specifies capability as parameter
    - Possession of capability means access is allowed
  - Capability list associated with domain but never directly accessible by domain
    - Rather, protected object, maintained by OS and accessed indirectly
    - Like a "secure pointer"
    - Idea can be extended up to applications

Implementation of Access Matrix (Cont.)

- Option 4 – Lock-key
  - Compromise between access lists and capability lists
  - Each object has list of unique bit patterns, called **locks**
  - Each domain as list of unique bit patterns called **keys**
  - Process in a domain can only access object if domain has key that matches one of the locks

Comparison of Implementations

- Many trade-offs to consider
  - Global table is simple, but can be large
  - Access lists correspond to needs of users
    - Determining set of access rights for domain non-localized so difficult
    - Every access to an object must be checked
      - Many objects and access rights -> slow
  - Capability lists useful for localizing information for a given process
    - But revocation capabilities can be inefficient
  - Lock-key effective and flexible, keys can be passed freely from domain to domain, easy revocation
- Most systems use combination of access lists and capabilities
  - First access to an object -> access list searched
    - If allowed, capability created and attached to process
      - Additional accesses need not be checked
    - After last access, capability destroyed
    - Consider file system with ACLs per file

Revocation of Access Rights

- Various options to remove the access right of a domain to an object
  - Immediate vs. delayed
  - Selective vs. general
  - Partial vs. total
  - Temporary vs. permanent
- **Access List** – Delete access rights from access list
  - Simple – search access list and remove entry
  - Immediate, general or selective, total or partial, permanent or temporary

Revocation of Access Rights

- **Capability List** – Scheme required to locate capability in the system before capability can be revoked
  - Reacquisition – periodic delete, will reaquire and denial if revoked
  - Back-pointers – set of pointers from each object to all capabilities of that object (MULTICS)
  - Indirection – capability points to global table entry which points to object – delete entry from global table, not selective (CAL)
  - Keys – unique bits associated with capability, generated when capability created
    - Master key associated with object, key matches master key for access
    - Revocation – create new master key
    - Policy decision of who can create and modify keys – object owner or others?

Program Threats

- Many variations, many names
- **Trojan Horse:** It is a code that is malicious in nature and has the capacity to take control of the computer. It is designed to steal, damage, or do some harmful actions on the computer.
  - Code segment that misuses its environment
  - Exploits mechanisms for allowing programs written by users to be executed by other users
  - **Spyware**, **pop-up browser windows**, **covert channels**
  - Up to 80% of spam delivered by spyware-infected systems
- **Trap Door**
  - A trap door is kind of a secret entry point into a program that allows anyone to gain access to any system without going through the usual security access procedures.
  - Specific user identifier or password that circumvents normal security procedures

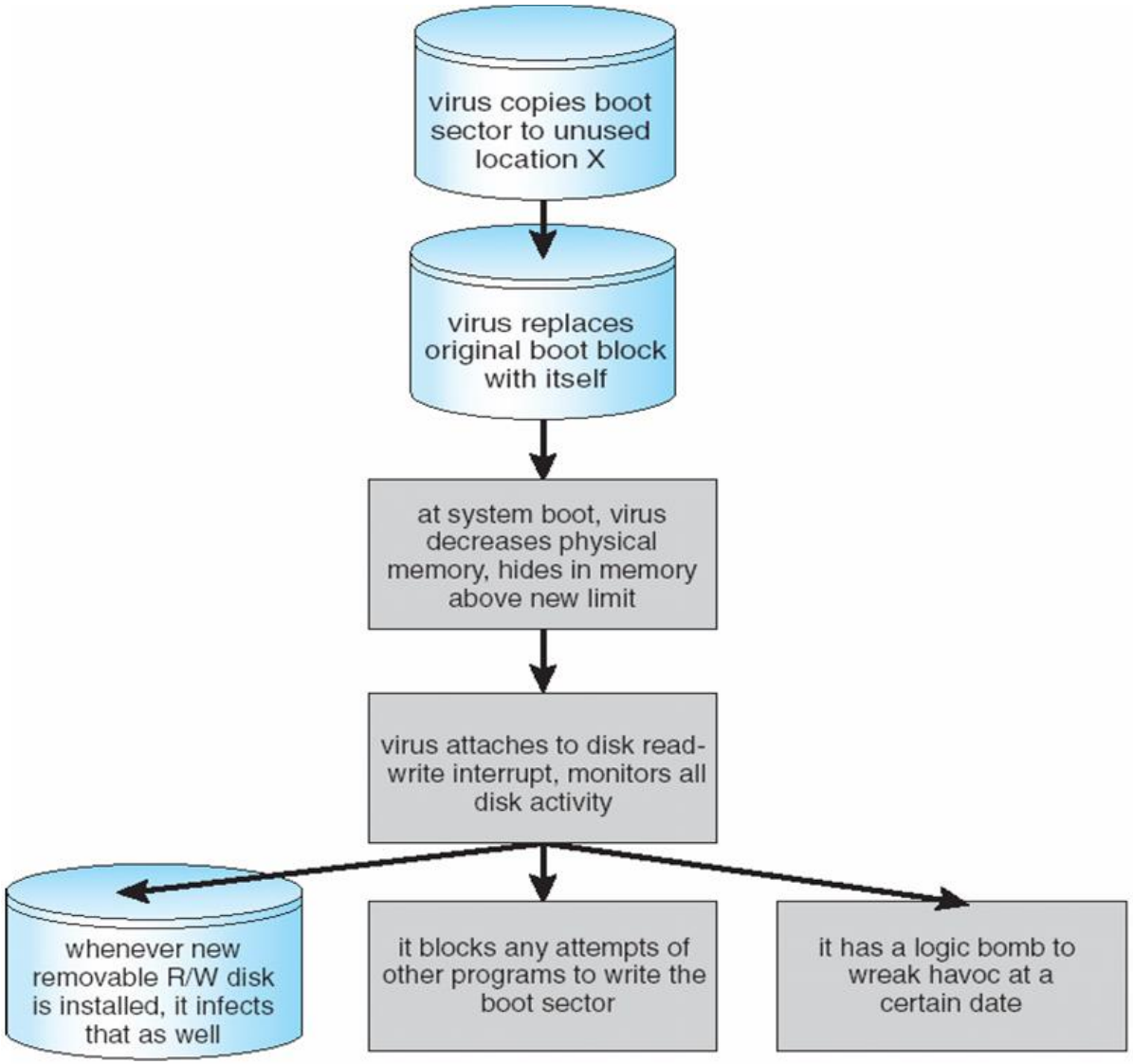
Program Threats (Cont.)

- **Logic Bomb**
  - A logic bomb is a type of malware designed to attack computer systems. It is code that is placed in a program and executes a specific set of instructions when certain conditions are met.
  - Program that initiates a security incident under certain circumstances
- **Stack** and **Buffer Overflow**
  - Exploits a bug in a program (overflow either the stack or memory buffers)
  - Failure to check bounds on inputs, arguments
  - Write past arguments on the stack into the return address on stack
  - When routine returns from call, returns to hacked address
    - Pointed to code loaded onto stack that executes malicious code
  - Unauthorized user or privilege escalation

Program Threats (Cont.)

- **Virus: A Virus is a malicious executable code attached to another executable file which can be harmless or can modify or delete data.**
- Many categories of viruses, literally many thousands of viruses
  - File
  - Boot
  - Macro
  - Source code
  - Polymorphic
  - Encrypted
  - Stealth
  - Tunneling
  - Multipartite
  - Armored

A Boot-sector Computer Virus



System and Network Threats

- **Worms**

  A Worm is a form of malware that replicates itself and can spread to different computers via Network.

  – use spawn mechanism; standalone program
- Internet worm
  - Exploited UNIX networking features (remote access) and bugs in *finger* and *sendmail* programs
  - Grappling hook program uploaded & main worm program
- **Port scanning**
  - Automated attempt to connect to a range of ports on one or a range of IP addresses
- **Denial of Service**
  - Overload the targeted computer preventing it from doing any useful work
  - Distributed denial-of-service (DDOS) come from multiple sites at once